

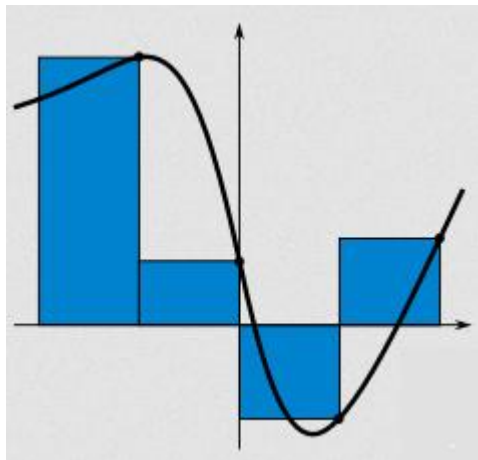
If you've taken a calculus class, you remember the definition of an integral. Simply put, it's the area under a curve¹. You may also recall that some functions are very difficult to integrate, primarily because of the challenge of taking the antiderivative. Polynomials are easy, and common trigonometric functions are not too difficult either. However, despite more advanced techniques such as integration by parts or integration by substitution, there remain functions that are not readily integrable. Fortunately, where exact techniques fail, we have one that will always work – a numeric approximation of an integral over a range. In these tasks, we'll explore that approach.

Task 1:

Define a function that we'll take the integral of. For now, choose something relatively simple, such as $x^2 - x$. Your function should have a single parameter, x , and return the corresponding y value at the given input.

Task 2:

We can readily approximate the area under a curve by modelling it as a number of rectangles, usually of equal width, as illustrated in the following image:



To do so:

- Split the range that we are trying to integrate into n equal-width subintervals.

¹ Technically, it's the *signed area* – you can have positive areas, where the curve is above the x -axis, negative areas, where it is below it, or both, if it crosses the axis. See the picture provided.

- For each subinterval, establish a point for which we will calculate the value of our function. Three common approaches are the *left endpoint* (the left, i.e. start, of the range), *right endpoint* (the right, i.e. end, of the range) or *centre point* (halfway between the left and right endpoints)
- For each representative point (established in the previous step), calculate the value of the function we are integrating at that point. This represents the height of the box at that interval
- The area of each box can then be trivially calculated from the width of the interval (which you can easily calculate from the total width, and the number of subintervals) and the height calculated previously
- The approximation of the integral is the sum of all of the calculated areas

Based on the algorithm above, write a function `numeric_integration(lower, upper, num_intervals)`. Your function should return the approximation for the integral that it calculated.

Task 3:

Write a function, `pct_error(actual_value, estimated_value)` to calculate the percentage error between a known-correct value and an estimated value. Your function should return a value between 0 and 1; for instance, 8.3 per cent would be .083.

Task 4:

Using Wolfram Alpha, or dusting off your calculus skills, calculate the integral of your chosen function (from Task 1) over a desired range.

Write a function, `integral_convergence(actual_value, max_intervals)` that calculates, and displays, how our estimate for the value of an integral converges to the correct/actual value.

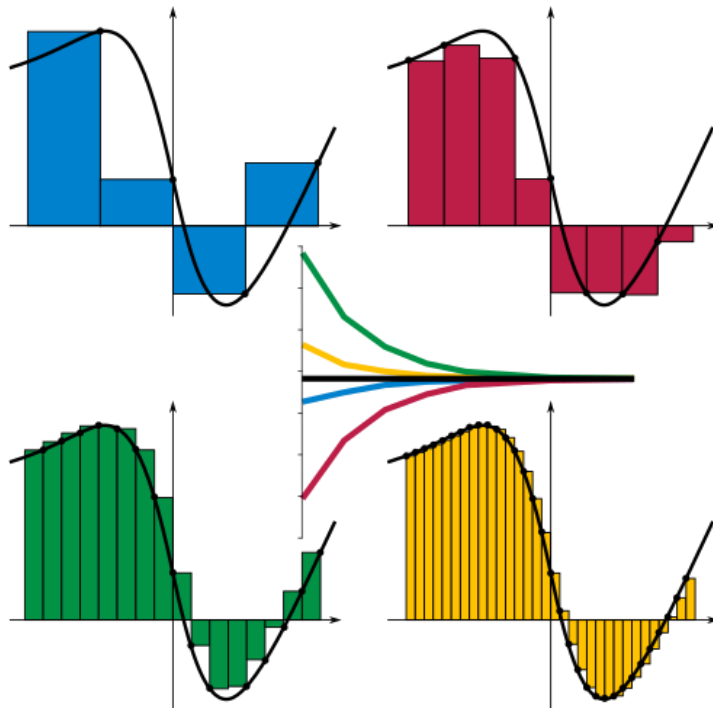
Your function should call your `numeric_integration` function from Task 2 repeatedly, passing in the same range that you used when calculating the actual value of the integral. It should start with a single interval, and go up to (and including) the `max_intervals` specified. For each iteration, estimate the value of the integral using your `numeric_integration` function; then, calculate the percentage error using your `pct_error` function.

Finally, print the number of intervals, estimated value, and percentage error. The following shows a sample output:

With 1 intervals, estimated value = 8.00, error = 40.00%
With 2 intervals, estimated value = 12.00, error = 10.00%
With 3 intervals, estimated value = 12.74, error = 4.44%
With 4 intervals, estimated value = 13.00, error = 2.50%
With 5 intervals, estimated value = 13.12, error = 1.60%
With 6 intervals, estimated value = 13.19, error = 1.11%
With 7 intervals, estimated value = 13.22, error = 0.82%
With 8 intervals, estimated value = 13.25, error = 0.62%
With 9 intervals, estimated value = 13.27, error = 0.49%
With 10 intervals, estimated value = 13.28, error = 0.40%

Because your function is printing its output, it doesn't need to return anything.

You should see that the more intervals that are used, the closer your estimate comes to the exact value, as illustrated in the following image:



Task 5:

Write a function `minimum_intervals(actual_value, max_intervals)` that will determine the minimum number of intervals that are necessary to get an answer within 1% of the true value specified.

You may employ a naïve strategy (perform the approximation using one interval, compare against actual, if too high, try two intervals, compare, ...) or a clever approach (for instance, something resembling the binary search algorithm we used when we played the guessing game previously). Your function should return the minimum number of intervals necessary, or `max_intervals+1` if it was not possible to do so with the number specified.

You should be able to reuse your `numeric_integration()` and `pct_error()` functions to delegate much of the work.

(Optional) Task 6:

If you're feeling up for a challenge, we can use a package to calculate the integral for us, rather than needing to defer to a third-party service and then manually enter the result. You can find some documentation [here](#).